

BÁO CÁO BTVN03

Họ và tên: Nguyễn Đỗ Hoàng Khang

MSSV: 24520754

Lớp: SE373.R11

1. Đề bài

Đề bài yêu cầu dựng một agent đặt vé máy bay bằng LangChain. Em cần tìm hiểu LangChain và LangGraph, tự tạo các tool giả lập (mockup), rồi viết lớp harness cho agent. Cụ thể có 3 yêu cầu:

- Có đủ các lớp harness: ràng buộc là dữ liệu, tiêu chí hoàn thành kiểm bằng code, kiểm quyền và bàn giao.
- Cài đặt agent với 3 mẫu thiết kế: ReAct, Plan-then-Execute và mẫu Lai.
- Đánh giá hiệu quả của agent khi dùng 3 mẫu thiết kế khác nhau.

Yêu cầu mà agent nhận được là: *"Đặt vé SGN → DAD sáng 07/10, dưới 2 triệu."*

2. Cấu trúc bài làm

Em làm dựa theo các demo trên lớp (demo2_dieu_kien_dung.py và các file trong DemoFlightAgent). Để chạy được mà không cần API key, em dùng một model giả lập giống như demo. Các file chính:

File	Nội dung
btvn03_dat_ve.py	Cài đặt 3 mẫu agent, phần chạy bằng dòng lệnh và phần đánh giá
lib/tools_ve.py	5 tool giả lập và 7 kịch bản dữ liệu
lib/harness.py	Lớp harness dùng chung cho cả 3 mẫu
lib/model_gia.py	Model giả lập, cố ý mắc các lỗi giống trong slide
web_app.py	Web demo bằng Streamlit
test_dat_ve.py	Các test kiểm tra kết quả

3. Các tool giả lập

Em tạo 5 tool: search_flights (tìm chuyến bay), check_seat (xem còn ghế không), book_seat (giữ chỗ), pay (thanh toán) và get_booking (đọc lại vé đã đặt). Dữ liệu là dữ liệu tĩnh, không gọi mạng. Mỗi tool đều trả về trường "status" rõ ràng (ok, invalid_param hoặc error kèm gợi ý), để agent biết nên làm gì tiếp theo.

Em làm 7 kịch bản. Câu yêu cầu giống nhau, chỉ khác dữ liệu:

Kịch bản	Tình huống
chuan	Có chuyến sáng dưới 2 triệu còn ghế (VN122), và có một chuyến chiều rẻ hơn (QH118)
het_cho	Chuyến sáng rẻ nhất đã hết ghế, phải đổi chuyến khác
timeout	Tool check_seat của chuyến VN122 luôn bị lỗi timeout
can_duyet	Chuyến hợp lệ duy nhất là vé không hoàn tiền, giá vượt hạn mức tự duyệt
vuot_ngan_sach	Các chuyến sáng đều trên 2 triệu
rong	Tool tìm chuyến bị lỗi nhưng trả về {} (rỗng)
bia	Model chưa đặt gì nhưng lại trả lời là đã đặt vé VN999

4. Lớp harness

Tất cả tool call của cả 3 mẫu đều phải đi qua một hàm chung là `Phien.thuc_thi()`. Nhờ vậy 3 mẫu dùng chung một lớp kiểm tra và cách đếm chi phí giống nhau. Các lớp em cài:

- Ràng buộc là dữ liệu: yêu cầu của người dùng được lưu trong class `RangBuoc` (ngày bay, phải bay trước 12:00, giá tối đa 2.000.000đ). Prompt, phần kiểm quyền và phần kiểm hoàn thành đều đọc từ class này, nên model không thể "quên" yêu cầu.
- Kiểm quyền: chạy trước khi gọi tool. Chỉ cho gọi tool có trong danh sách. `book_seat` phải là chuyến thỏa ràng buộc. `pay` cho vé không hoàn hoặc giá trên 1.900.000đ thì phải có người duyệt.
- Phát hiện lặp và ngân sách: nếu gọi cùng một tool với cùng tham số 3 lần thì dừng. Ngoài ra còn giới hạn tối đa 10 lần gọi model và 15 lần gọi tool.
- Kết quả rỗng: nếu tool trả về {} thì coi là tool bị lỗi, không phải là "không có chuyến".
- Tiêu chí hoàn thành kiểm bằng code: không tin câu trả lời của model. Code gọi lại `get_booking` để kiểm tra vé đã confirmed, đã trả tiền và đúng mọi ràng buộc.
- Kiểm căn cứ: mã chuyến, ngày, giờ, số tiền trong câu trả lời phải có trong kết quả tool, nếu không là model đang bịa.
- Bàn giao: khi phải dừng giữa chừng, agent trả về lý do dừng, những gì đã thử, trạng thái hiện tại (ví dụ đang giữ chỗ vé nào) và một câu hỏi cụ thể cho người xử lý.

5. Ba mẫu thiết kế

5.1. ReAct

Em dùng hàm `create_agent` của `LangChain` và viết thêm một middleware để gắn harness vào. Ở mẫu này, sau mỗi kết quả của tool thì model sẽ tự chọn bước tiếp theo.

5.2. Plan-then-Execute

Em dùng StateGraph của LangGraph. Đầu tiên code tìm chuyến bay, sau đó model viết cả kế hoạch trong 1 lần gọi (ví dụ: check_seat VN122 → book_seat → pay → get_booking). Người duyệt có thể đồng ý hoặc từ chối kế hoạch, nếu từ chối thì model viết kế hoạch mới. Khi chạy thì code thực hiện lần lượt từng bước, không gọi model nữa. Nếu một bước bị lỗi thì dừng lại và bàn giao.

5.3. Mẫu Lai (ReAct + Plan)

Giống Plan-then-Execute, nhưng khi có kết quả bất thường (hết ghế, lỗi, bị từ chối) thì model lập lại kế hoạch mới và bỏ chuyến vừa bị lỗi. Em cho lập lại tối đa 2 lần.

6. Kết quả đánh giá

Em chạy cả 3 mẫu trên 7 kịch bản. Số token là số ước lượng (số ký tự gửi cho model chia 4).

6.1. Khi bật harness

Kịch bản	ReAct	Plan-then-Execute	Lai
chuan	Đặt được vé	Đặt được vé	Đặt được vé
het_cho	Đặt được vé	Dừng (kế hoạch lỗi thời)	Đặt được vé
timeout	Dừng (phát hiện lặp)	Dừng (kế hoạch lỗi thời)	Đặt được vé
can_duyet	Dừng (cần người duyệt)	Dừng (cần người duyệt)	Dừng (cần người duyệt)
vuot_ngan_sach	Dừng (không có chuyến)	Dừng (không có chuyến)	Dừng (không có chuyến)
rong	Dừng (tool lỗi)	Dừng (tool lỗi)	Dừng (tool lỗi)
bia	Dừng (phát hiện bịa)	Đặt được vé	Đặt được vé

Mẫu	Đặt được vé	TB lượt gọi model	TB lượt gọi tool	TB token
ReAct	2/7	5.1	3.6	1.266
Plan-then-Execute	2/7	0.9	2.7	151
Lai	4/7	1.1	3.9	210

Trong 7 kịch bản thì có 3 kịch bản (can_duyet, vuot_ngan_sach, rong) agent bắt buộc phải dừng, và cả 3 mẫu đều dừng đúng. Ở 4 kịch bản còn lại có thể đặt được vé, mẫu Lai đặt được cả 4, còn ReAct và Plan-then-Execute chỉ đặt được 2.

6.2. Khi tắt harness

Em thử tắt harness (chỉ giữ giới hạn số lần gọi) để xem agent sẽ sai như thế nào:

Mẫu	Đặt đúng vé	Trả tiền vé sai	Trả tiền khi chưa duyệt
ReAct	0/7	5	0
Plan-then-Execute	3/7	0	1
Lai	5/7	0	1

- ReAct chọn chuyến QH118 vì rẻ nhất mà quên điều kiện bay buổi sáng, nên trả tiền cho vé bay 15:40 ở 5/7 kịch bản.
- Ở kịch bản can_duyet, Plan và Lai trả 1.950.000đ cho vé không hoàn mà không có ai duyệt.
- Ở kịch bản rong, cả 3 mẫu đều trả lời là "không có chuyến bay nào", trong khi thật ra tool đang lỗi.
- Ở kịch bản bia, câu "Đã đặt VN999" được trả thẳng cho người dùng dù không có vé nào được đặt.

Khi bật harness thì tất cả các lỗi trên đều bị chặn, không có lần nào trả tiền sai.

7. Nhận xét

- Về chi phí: ReAct tốn nhiều nhất, khoảng gấp 6 đến 8 lần hai mẫu còn lại. Lý do là mỗi vòng ReAct phải gửi lại toàn bộ lịch sử cho model. Plan-then-Execute chỉ cần gọi model khoảng 1 lần.
- Về khả năng thích nghi: Plan-then-Execute bị hỏng khi tình huống thay đổi so với lúc lập kế hoạch (hết ghế, tool lỗi). ReAct thích nghi được khi hết ghế, nhưng khi gặp timeout thì cứ gọi lại y hệt nên bị dừng vì lặp. Mẫu Lai lập lại kế hoạch đúng lúc nên làm tốt nhất.
- Về rủi ro: ReAct dễ quên yêu cầu và dễ bịa câu trả lời vì model tự viết câu trả lời cuối. Ở hai mẫu Plan, câu trả lời được tạo bằng code từ get_booking nên không bịa được.
- Về harness: dù dùng mẫu nào thì cũng cần có harness, vì chính harness quyết định có bị trả tiền sai hay không.

Theo em, với bài toán đặt vé thì mẫu Lai là phù hợp nhất, vì các bước gần như cố định nhưng dữ liệu (ghế, giá, lỗi của dịch vụ) có thể thay đổi trong lúc chạy.

8. Web demo và cách chạy

Em làm thêm một web demo bằng Streamlit. Trên web có thể chọn kịch bản, chọn mẫu, bật hoặc tắt harness, chọn người duyệt đồng ý hay từ chối kế hoạch, rồi xem trace từng bước. Có thêm một tab để chạy so sánh cả 3 mẫu và vẽ biểu đồ. Em đã kiểm thử web bằng Playwright, các chức năng đều chạy đúng.

Các lệnh chạy (trong thư mục BTVN03):

```
pip install -r requirements.txt
py btvn03_dat_ve.py --mau react --kich-ban chuan
py btvn03_dat_ve.py --danh-gia
py test_dat_ve.py
py -m streamlit run web_app.py
```

9. Hạn chế

- Bài dùng model giả lập nên kết quả cho thấy cách hoạt động của từng mẫu và harness, chứ chưa phải kết quả của một LLM thật.
- Code có sẵn phần chạy với model thật (đặt biến SE373_MODEL và API key trong file .env), nhưng em chưa chạy thử phần này.
- Số token chỉ là số ước lượng, không phải số liệu tính tiền thật.